Database Management System

Unit-3

The Relational Algebra

and

NoSQL Databases

Chapter-1

The Relational Algebra

Outline

- Introduction
- Unary Relational Operations: SELECT, PROJECT.
- Relational Algebra Operations from Set Theory.
- Binary Relational Operations: JOIN, DIVISION.
- Additional Relational Operations.
- Examples of Queries in Relational Algebra

Introduction

- **Relational Algebra(RA)** came in 1970 and given by EF code father of DBMS.
- RA is also called **procedural language** or **formal query language.**
- In procedural or formal query language programmer or user who is writing query has to mentions two things:
  - What to do? and
  - How to do?
- RA is like C-programming, we have to mention what we actually want to do plus how to do that work. This is the meaning of procedural language.
- RA came in 1970's!!! **Why is it important? What is its value in today's time?**

Introduction

- Actually what we use in today's time that is SQL, it is the base of relational algebra.
- RA is the collection of mathematical expression.
- By using various mathematical expressions a theoretical model is being made to show how to access data.
- But that mathematical expression model is used to make SQL.
- We use SQL, Oracle, Microsoft sequel server, IBM DB2 whatever platform we are using, what is used there? SQL!! But who has helped in making this SQL?
- The base of all the technology of accessing data is Relational Algebra.

Introduction

Operators

Basic Operators

- ❖ **Projection (π)**
- ❖ **Selection (σ)**
- ❖ **Cross Product (X)**
- ❖ **Union (U)**
- ❖ **Rename (ρ)**
- ❖ **Set Difference (-)**

Derived Operators

- ❖ **Join ($\bowtie$)**
- ❖ **Intersect ($\cap$)**

  $(X \cap Y) = X-(X-Y)$
- ❖ **Division (/, ÷)**

Projection in RA

- Projection (π) operator.
- What is the use of projection operation?

Student

RollNo	Name	Age
1	A	20
2	B	21
3	A	19

- **What is the function of π operator?**
  - **Retrieving the data.**
  - Whenever we have to fetch the data from a particular column, it can be one or more than one column, then we will have to use **π projection.**
- **Query1:** Retrieve the RollNo from the table Student.

Projection in RA cntd..

- **<u>To fetch the RollNo, how we write in RA is:</u>**

$$\pi_{RollNO}(Student)$$

- π means project. Project means from the whole table, extract only the RollNo (as per query)
- **Query2**: Retrieve the RollNo and Name from the table Student.
- **<u>To fetch the RollNo and Name, how we write in RA is:</u>**

$$\pi_{RollNO,Name}(Student)$$

RollNo	Name
1	A
2	B
3	A

Table: Result of the above Query2

Projection in RA cntd..

- The Projection (π) operator always works in **DISTINCT** format.
- Distinct means unique.
- Ex: π_{Name} **(Student)** ⟹

Name
A
B

- Projection (π) operator by default will give unique answer, it wont show duplicate data.
- Fetching RollNo and Name wont be a problem in the above table student because in that table 1,A and 3,A are different(unique). They are not duplicate.
- Projection we always write in the last, before that we write other operators.

Selection in RA

- Selection (σ) operator.
- What is the use of Selection operation?

Student

RollNo	Name	Age
1	A	20
2	B	21
3	A	19

- **What is the function of σ** operator?
  - **Selecting the data.**
  - Whenever we have to select the data from a particular row(tuple), then we will have to use **σ operator**
- **Query1:** Retrieve the Name of the student whose RollNo=2.

Selection in RA cntd..

- Finally what I have to show as per the query?
  - ✓ Retrieve the Name.
- Retrieve means π (we have to apply projection operation).
- But projection operation we always apply at last. So first select the data as per the condition given in the query:

$$\sigma_{RollNo=2}\ (Student)$$

Output of this is 2, B, 21

$$\pi_{Name}(\sigma_{RollNo=2}\ (Student))$$

→FinalOutput

- **"Selection works on rows(tuples). Projection works on columns(attributes)"**

Selection in RA cntd..

- Say suppose I do reverse:

$$\sigma_{RollNo=2}\ (\pi_{Name}(Student))$$

Output→ $\sigma_{RollNo=2}$ (A,B)

- **"Selection operator first extracts the desired tuple. From that tuple what output should be shown will be projected by projection operation"**

Unary Relational Operations: SELECT, PROJECT

1. The Select Operation

- The SELECT operation is used to choose a *subset* of the tuples from a relation that satisfies a **selection condition.**
- We can consider the SELECT operation to be a ***filter*** that keeps only those tuples that satisfy a qualifying condition.
- The SELECT operation can also be visualized as a ***horizontal partition*** of the relation into two sets of tuples—those tuples that satisfy the condition and are selected, and those tuples that do not satisfy the condition and are filtered out.
- For example, to select the EMPLOYEE tuples whose department is 4, or those whose salary is greater than $30,000, we can individually specify each of these two conditions with a SELECT operation as follows: $\sigma_{Dno=4}(EMPLOYEE)$ $\sigma_{Salary>30000}(EMPLOYEE)$

13

Unary Relational Operation: SELECT

- In general, the SELECT operation is denoted by

$$\sigma_{<\text{selection condition}>}(R)$$

- where the symbol σ (sigma) is used to denote the SELECT operator and the selection condition is a Boolean expression (condition) specified on the attributes of relation R.
- The Boolean expression specified in <selection condition> is made up of a number of **clauses** of the form:

<attribute name> <comparison op> <constant value>

or

<attribute name> <comparison op> <attribute name>

14

Unary Relational Operation: SELECT

- where <attribute name> is the name of an attribute of R, <comparison op> is normally one of the operators $\{=, <, \leq, >, \geq, \neq\}$, and <constant value> is a constant value from the attribute domain.
- Clauses can be connected by the standard Boolean operators *and*, *or*, and *not* to form a general selection condition.
- For example, to select the tuples for all employees who either work in department 4 and make over$25,000 per year, or work in department 5 and make over $30,000, we can specify the following SELECT operation: $\sigma_{(Dno=4\ \textbf{AND}\ Salary>25000)\ \textbf{OR}\ (Dno=5\ \textbf{AND}\ Salary>30000)}(EMPLOYEE)$

  The result is shown in Figure(a).

(a)

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5

Unary Relational Operation: SELECT

- The SELECT operator is **unary**; that is, it is applied to a single relation.
- Moreover, the selection operation is applied to *each* ***tuple individually***; hence, selection conditions cannot involve more than one tuple.
- The **degree** of the relation resulting from a SELECT operation its number of attributes—is the same as the degree of R.
- The number of tuples in the resulting relation is always ***less than or equal*** *to* the number of tuples in R.
- The fraction of tuples selected by a selection condition is referred to as the **selectivity** of the condition.

Unary Relational Operation: SELECT

In SQL, the SELECT condition is typically specified in the *WHERE clause* of a query. For example, the following operation:

$\sigma_{Dno=4\ AND\ Salary>25000}$ (EMPLOYEE)

would correspond to the following SQL query:

```
SELECT  *
FROM    EMPLOYEE
WHERE   Dno=4 AND Salary>25000;
```

17

Unary Relational Operations: SELECT, PROJECT

2. The Project Operation

- If we think of a relation as a table, the SELECT operation chooses some of the ***rows*** from the table while discarding other rows.
- The **PROJECT** operation, on the other hand, selects certain ***columns*** from the table and discards the other columns.
- If we are interested in only certain attributes of a relation, we use the PROJECT operation to ***project*** the relation over these attributes only. Therefore, the result of the PROJECT operation can be visualized as a ***vertical partition*** of the relation into two relations:
  - One has the needed columns (attributes) and contains the result of the operation, and the other contains the discarded columns. For example, to list each employee's first and last name and salary, we can use the PROJECT operation as follows:

18

Unary Relational Operation: PROJECT

$$\pi_{\text{Lname, Fname, Salary}}(\text{EMPLOYEE})$$

The resulting relation is shown in Figure(b).

(b)

Lname	Fname	Salary
Smith	John	30000
Wong	Franklin	40000
Zelaya	Alicia	25000
Wallace	Jennifer	43000
Narayan	Ramesh	38000
English	Joyce	25000
Jabbar	Ahmad	25000
Borg	James	55000

Unary Relational Operation: PROJECT

- The general form of the **PROJECT** operation is $\pi_{\text{<attribute list>}}(R)$
- where π (pi) is the symbol used to represent the PROJECT operation, and <attribute list> is the desired sub list of attributes from the attributes of relation R.
- The result of the PROJECT operation has only the attributes specified in <attribute list> *in **the same order as they appear in the list***. Hence, its **degree** is equal to the number of attributes in <attribute list>.
- If the attribute list includes only nonkey attributes of R, duplicate tuples are likely to occur. The PROJECT operation *removes any duplicate tuples*, so the result of the PROJECT operation is a set of distinct tuples, and hence a valid relation. This is known as **duplicate elimination**.

Relational Algebra Operations from Set Theory

Binary Relational Operations: Join and Division

Binary Relational Operations: Join

- In this section we are going to learn binary relational operation: **Join**.

1. The Join Operation(⋈)

- The JOIN operation, denoted by ⋈, is used to combine related tuples from two relations into single "longer" tuples based on a given join condition.
- This operation is very important for any relational database with more than a single relation because it allows us to process relationships among relations.

Syntax: $R \bowtie_{\text{<join condition>}} S$

Binary Relational Operations: Join

Example:

DEPARTMENT	DName	DNo	Mgr_SSN
	Research	2	553621425
	Finance	5	996856974

EMPLOYEE	SSN	FName	LName	DNo
	123658974	Alex	Smith	2
	553621425	Fred	Scott	5
	996856974	Elsa	David	5
	859689742	Peter	Williams	2

- Now If I want to retrieve the details of the manager of each department, I have to combine each department tuple with the employee tuples and then check if their SSN values match.
- For this I have to join two relations the department relation and the employee relation by a **join condition that is manager SSN is equal to SSN** and then I will get the details of the manager of each department.

Binary Relational Operations: Join

$$Dept_Mgr \leftarrow DEPARTMENT \bowtie_{Mgr_SSN = SSN} EMPLOYEE$$

- This join operation is nothing but Cartesian product followed by the selection operation that is instead of join operation, I can specify the Cartesian product followed by the selection operation.

Let us see how that works:

$$Temp \leftarrow DEPARTMENT \times EMPLOYEE$$
$$Dept_Mgr \leftarrow \sigma_{(MgrSSN = SSN)}(Temp)$$

- "Cartesian product followed by the selection operation can be used instead of join operation" and it will give the same result.

Binary Relational Operations: Join

- The only difference between the cartesian product and the join operation is that in join only the combination of tuples that satisfy the join condition is displayed in the result.
- Whereas, in the cartesian product all the combination of tuples from the two relations are specified in the result.

Let us see the result of join operation and how that works:

DEPARTMENT

DName	DNo	Mgr_SSN
Research	2	553621425
Finance	5	996856974

EMPLOYEE

SSN	FName	LName	DNo
123658974	Alex	Smith	2
553621425	Fred	Scott	5
996856974	Elsa	David	5
859689742	Peter	Williams	2

Binary Relational Operations: Join

- The first tuple from the department relation combines with every tuple from the employee relation.
- Similarly, second tuple from the department relation combines with every tuple from the employee relation and from these combined tuples we select only those tuples where manager SSN is equal to SSN.

Dept_Mgr	DName	DNo	Mgr_SSN	SSN	FName	LName	DNo
	Research	2	333621425	333621425	Fred	Scott	2
	Finance	5	996356974	996356974	Elsa	David	5

Result of the JOIN operation

Binary Relational Operations: Join

- Here the first tuple from the department relation combines with the first tuple from the employee relation and then the condition is checked if manager SSN is equal to SSN. Since it is not equal that combined tuple will not be displayed.
- Again the first tuple of the department relation is combined with the second tuple of the employee relation and the condition is checked that is if manager SSN is equal to SSN. Since it is equal that combined tuple will be displayed an so on.

Binary Relational Operations: Join

❑ The Theta Join (θ)

- In Join operation, we mention about the join operator followed by the join condition.
- Now that join condition is of the form:

<join condition> : A_i θ B_j

Where,

A_i ⇒ Attribute of R
B_j ⇒ Attribute of S
θ ⇒ { =, <, <=, >, >=, ≠ }

Syntax: $R \bowtie_{A_i \theta B_j} S$

29

Binary Relational Operations: Join

- Let me take a simple example to demonstrate the theta join:

R	A_1	A_2
	20	25
	80	40

S	B_1
	50
	35

- On the above two relations we can do a join operation with different comparison operators like:

$R \bowtie_{A_2 = B_1} S$ | $R \bowtie_{A_2 > B_1} S$
$R \bowtie_{A_2 < B_1} S$ | $R \bowtie_{A_2 >= B_1} S$
$R \bowtie_{A_2 <= B_1} S$ | $R \bowtie_{A_2 \neq B_1} S$

These are the different operators we can use on the attributes of both the relations.

30

Binary Relational Operations: Join

- **Let me take a simple example to demonstrate the theta join and see its result:**

$$R \bowtie_{A_2 > B_1} S$$

R	A1	A2
	20	25
	80	40

S	B1
	50
	35

A1	A2	B1
20	25	50
20	25	35
80	40	50
80	40	35

Result of "X"

A1	A2	B1
80	40	35

Result of "⋈"

✓ Theta Join allows us to join or merge two relation based on the condition represented by theta and theta join works for all the comparison operators.

Binary Relational Operations: Join

- **Next we look into the few variations of joins:**

❑ The EQUIJOIN Operation

- The only comparison operator used is '=' are called equijoin.

Example:

$$Dept_Mgr \leftarrow DEPARTMENT \bowtie_{Mgr_SSN = SSN} EMPLOYEE$$

❑ The Natural Join Operation (*)

- In equijoin it is joining of two relations based on the given condition.
- In the above example, in the given condition it is manager SSN is equal to SSN we have already seen the result.

Binary Relational Operations: Join

Dept_Mgr	DName	DNo	Mgr_SSN	SSN	FName	LName	DNo
	Research	2	553621425	553621425	Fred	Scott	2
	Finance	5	996856974	996856974	Elsa	David	5

Result of the JOIN operation

- In this result we see that values manager SSN and SSN are same in every tuple of this relation because of the equality join condition that we specified on these two (Mgr_SSN and SSN) attribute that is Mgr_SSN should be equal to SSN.
- Since the values of both the attributes are same and therefore not necessary, a new operation called the natural join operation was created **to get rid of this repetition or duplication** when it comes to equijoin.

33

Binary Relational Operations: Join

- Now in natural join the two attributes by which we are joining two relations or in other words the two join attributes must have the same name in both the relations we are trying to join.
- That is, Natural join can be performed **only if there is a common attribute** in between the relations.

PROJECT	PID	PName	DNum
	101	ProjectX	1
	102	ProjectY	2
	103	ProjectZ	2

DEPARTMENT	DNo	Mgr_SSN
	1	553621425
	2	996856974

- In the above two relations the common attribute is department number but their names are different in both the relations.

34

Binary Relational Operations: Join

- To apply natural join on these two relations we have to first rename the common attribute so that they have the same name.
- Here we have to rename the DNo of the department relation to DNum to match with the name of the project relation.

$$Proj_Dept \leftarrow PROJECT * \rho_{(DNum,\ Mgr_SSN)}(DEPARTMENT)$$

- This same query can be written in a series of steps by creating an intermediate table.

$$DEPT \leftarrow \rho_{(DNum,\ Mgr_SSN)}(DEPARTMENT)$$
$$Proj_Dept \leftarrow PROJECT * DEPT$$

NOTE: If the common attributes of both the relations have the same name then we directly apply the natural join, we don't have to rename the attributes.

Binary Relational Operations: Join

- Now let us see the result of the above natural join operation.

Proj_Dept	PID	PName	DNum	Mgr_SSN
	101	ProjectX	1	553621425
	102	ProjectY	2	996856974
	103	ProjectZ	2	996856974

Result of the Natural Join (*) operation

"Natural join is basically a equijoin followed by the removal of unnecessary attributes."

- In the above relation DNum is the join attribute.
- We see in the resulting relation each project is combined with its controlling department.
- Natural join does not use any comparison operator and also it does not concatenate the way a cartesian product does.
- Natural join must have atleast one common attribute between the two relations and then it displays the result where the values of the common attributes are the same.

Binary Relational Operations: Division (÷)

- Example: To retrieve the employee ID (EID) of the employees working on **all** projects.
- **Here I am taking two relations:**

EMPLOYEE	EID	PID
	1001	1
	1002	1
	1002	2
	1003	2

PROJECT	PID
	1
	2

- From the above two relations I want to retrieve the employee ID (EID) of the employees working on **all** projects.
- For this we are using division operation.

37

Binary Relational Operations: Division

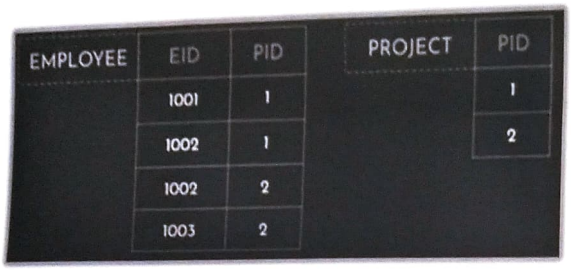
- **Note:** When we see the keyword **all** we need to use the division operation.
- Few examples are students enrolled in all the courses or persons having bank account in all the bank and so on.
- So, "whenever we encounter the keyword **all** we would use the division operation".

$$Res \leftarrow EMPLOYEE \div PROJECT$$

- Let us see the result of the above query:

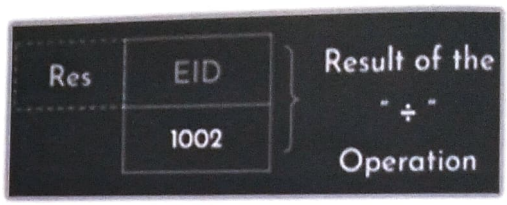
38

Binary Relational Operations: Division



EMPLOYEE	EID	PID
	1001	1
	1002	1
	1002	2
	1003	2

PROJECT	PID
	1
	2



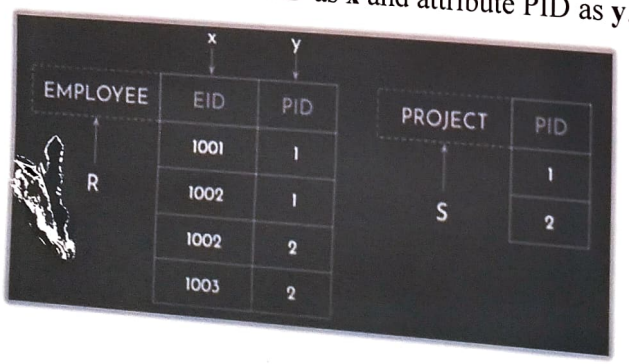
Res	EID
	1002

Result of the " ÷ " Operation

- Now let us see the result the query as series of steps using:
  - **Projection operator**
  - **Cartesian product and**
  - **Minus Operation**

Binary Relational Operations: Division

- For better understanding, I am considering employee relation as **R** and Project relation as **S.**
- Also we are considering attribute EID as **x** and attribute PID as **y**.



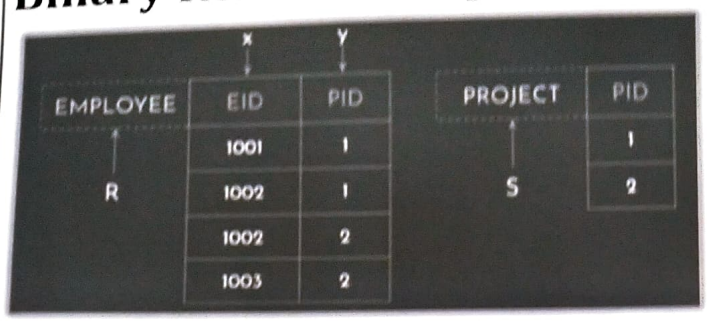
x y

EMPLOYEE	EID	PID
R	1001	1
	1002	1
	1002	2
	1003	2

PROJECT	PID
S	1
	2

- Now, let us see the series of steps

Binary Relational Operations: Division



$T1 \leftarrow \pi_{x}(R)$

$T2 \leftarrow \pi_{x}((T1 \times S) - R)$

$Res \leftarrow T1 - T2$

T1	EID
	1001
	1002
	1003

(T1 × S)

EID	PID
1001	1
1001	2
1002	1
1002	2
1003	1
1003	2

x

EID	PID
1001	2
1003	1

(T1 × S) - R

T2	EID
	1001
	1003

Res	EID
	1002

Table 8.1 Operations of Relational Algebra

OPERATION	PURPOSE	NOTATION
SELECT	Selects all tuples that satisfy the selection condition from a relation R.	$\sigma_{\text{<selection condition>}}(R)$
PROJECT	Produces a new relation with only some of the attributes of R, and removes duplicate tuples.	$\pi_{\text{<attribute list>}}(R)$
THETA JOIN	Produces all combinations of tuples from R_1 and R_2 that satisfy the join condition.	$R_1 \bowtie_{\text{<join condition>}} R_2$
EQUIJOIN	Produces all the combinations of tuples from R_1 and R_2 that satisfy a join condition with only equality comparisons.	$R_1 \bowtie_{\text{<join condition>}} R_2$, OR $R_1 \bowtie_{(\text{<join attributes 1>}), (\text{<join attributes 2>})} R_2$
NATURAL JOIN	Same as EQUIJOIN except that the join attributes of R_2 are not included in the resulting relation; if the join attributes have the same names, they do not have to be specified at all.	$R_1 *_{\text{<join condition>}} R_2$, OR $R_1 *_{(\text{<join attributes 1>}), (\text{<join attributes 2>})} R_2$ OR $R_1 * R_2$
UNION	Produces a relation that includes all the tuples in R_1 or R_2 or both R_1 and R_2; R_1 and R_2 must be union compatible.	$R_1 \cup R_2$
INTERSECTION	Produces a relation that includes all the tuples in both R_1 and R_2; R_1 and R_2 must be union compatible.	$R_1 \cap R_2$
DIFFERENCE	Produces a relation that includes all the tuples in R_1 that are not in R_2; R_1 and R_2 must be union compatible.	$R_1 - R_2$
CARTESIAN PRODUCT	Produces a relation that has the attributes of R_1 and R_2 and includes as tuples all possible combinations of tuples from R_1 and R_2.	$R_1 \times R_2$
DIVISION	Produces a relation $R(X)$ that includes all tuples $t[X]$ in $R_1(Z)$ that appear in R_1 in combination with every tuple from $R_2(Y)$, where $Z = X \cup Y$.	$R_1(Z) \div R_2(Y)$

Additional Relational Operations

- *In this section we are going to learn additional relational algebra operations.*
- *Some database requests specified by the database users cannot be performed with the original relational algebra operations.*
- *That is, it cannot be performed with the unary set theory or the binary relational algebra operations.*
- *In this section we define additional operations to express these requests.*
- *These operations enhance the expressive power of the original relational algebra.*

Additional Relational Operations

❑ **Generalised Projection**

- The generalized projection operation **extends the projection operation** by allowing functions of attributes to be included in the projection list.
- The generalized form can be expressed as $\pi_{F1, F2, ..., Fn}(R)$
- where $F_1, F_2, \ldots, F_n$ are functions over the attributes in relation R and may involve arithmetic operations and constant values.
- This operation is helpful when developing reports where computed values have to be produced in the columns of a query result.

Additional Relational Operations

As an example, consider the relation

EMPLOYEE (Ssn, Salary, Deduction, Years_service)

A report may be required to show

Net Salary = Salary − Deduction,
Bonus = 2000 * Years_service, and
Tax = 0.25 * Salary

Then a generalized projection combined with renaming may be used as follows:

$$\text{REPORT} \leftarrow \rho_{(\text{Ssn, Net_salary, Bonus, Tax})}(\pi_{\text{Ssn, Salary} - \text{Deduction, 2000} * \text{Years_service, 0.25} * \text{Salary}}(\text{EMPLOYEE}))$$

Additional Relational Operations

- **Aggregate Functions ℑ and Grouping**

- Another type of request that cannot be expressed in the basic relational algebra is to specify mathematical aggregate functions on collections of values from the database.
- Examples of such functions include retrieving the **average or total salary** of all employees or the total number of employee tuples.
- These functions are used in simple statistical queries that summarize information from the database tuples.
- Common functions applied to collections of numeric values include SUM, AVERAGE, MAXIMUM, and MINIMUM. The COUNT function is used for counting tuples or values.

Additional Relational Operations

❑ **Aggregate Functions ℱ and Grouping**

- Another common type of request involves **grouping the tuples** in a relation by the value of some of their attributes and then applying an aggregate function independently to each group.
- An example would be to group EMPLOYEE tuples by Dno, so that each group includes the tuples for employees working in the same department.

❖**We can define an AGGREGATE FUNCTION operation, using the symbol ℱ (pronounced script F)*, to specify these types of requests as follows:**

Syntax:

$$\langle grouping_attributes \rangle\ \mathcal{F}\ \langle function_list \rangle\ (R)$$

<grouping_attributes> ⇒ list of attributes in R

<function_list> ⇒ list of (<function><attribute>) pairs

Additional Relational Operations

- Grouping attributes are not mandatory and we can apply the aggregate functions without these grouping attributes.

Example1: To retrieve the number of employees and their average salary.

EMPLOYEE

FName	SSN	Salary	DNo
Ann	123654789	40000	2
Jeremy	969687423	30000	2
Peter	333265874	30000	1
Elsa	888548623	20000	2

Here we do not need to group by any attribute we can directly apply aggregate function

$$\mathcal{F}\ COUNT_{SSN},\ AVERAGE_{Salary}\ (EMPLOYEE)$$

Additional Relational Operations

- Now let us see the result of the above query

Result

COUNT_SSN	AVERAGE_Salary
4	30000

- When the attribute name of the resulting relation is not mentioned explicitly in the query, then in the result the attribute name would be function name followed by underscore and then followed by the attribute name on which we performed the function.

Additional Relational Operations

Example2: To retrieve the number of employees and their average salary in each department.

EMPLOYEE

FName	SSN	Salary	DNo
Ann	123654789	40000	2
Jeremy	969687423	30000	2
Peter	333265874	30000	1
Elsa	888548623	20000	2

- In the above relation we have two departments that is department-1 and department-2. So we need to count the number of employees in each department and also their average salary.

Since we need the result by each department we need to group by department.

So in this relation employee the attribute that contains information about the department is DNo. So DNo will be the grouping attribute.